

SIMULATING X-RAY SCAT-
TERING:

A PRACTICAL INTRO-
DUCTION

USING PYTHON AND GITHU

Contents

Preface 5

I Setting Up Your Scientific Computing Environment 7

Python and Package Management with uv 9

Version Control with Git and GitHub 13

VS Code and Jupyter Notebooks 17

Your First Commit 21

Remote Computing via SSH 25

Exercise Solutions 27

Preface

These notes are intended for undergraduate students and researchers early in their graduate careers who want to build practical computational skills while exploring the physics of X-ray scattering from soft-matter systems. The goal is deliberately modest: by the end of this guide you will be able to write clean, well-documented Python code that simulates how X-rays scatter from simple geometric objects—starting with a solid sphere and ending with a multi-shell hollow vesicle that captures the essential physics of a lipid bilayer membrane. Along the way you will also learn how to manage software environments, track your work with version control, and produce publication-quality figures.

No prior experience with scattering theory is assumed. A working knowledge of algebra and basic calculus will be helpful, as will some familiarity with programming in any language, though complete beginners to Python are welcome. Each section of this guide builds directly on the previous one, and it is strongly recommended that you work through them in order.

The exercises at the end of each chapter are an essential part of the learning process. Full worked solutions are collected in Appendix I, but you are strongly encouraged to attempt each problem before consulting them.

A note on software versions: This guide was written using Python 3.12 and uv 0.4. The commands shown should work without modification on macOS, Linux, and Windows (via PowerShell or WSL2), but minor differences may appear. Where platform-specific notes are needed they are marked clearly.

Part I

Setting Up Your Scientific Computing Environment

Python and Package Management with uv

Why Python?

Python has become the dominant language for scientific computing over the past decade, and for good reason. It is free and open-source, runs on every major operating system, and has an enormous ecosystem of libraries purpose-built for numerical work, data visualisation, and statistical analysis. For our purposes the most important of these are:

- **NumPy** — fast numerical arrays and mathematical functions
- **SciPy** — scientific algorithms including special functions and optimisation routines
- **Matplotlib** — publication-quality plotting
- **JupyterLab** — an interactive notebook interface that runs in your browser or directly inside VS Code

Python is also designed to be readable. The goal of this guide is for you to understand every line of code you write, not just copy it. Keeping code readable—with clear variable names and short explanatory comments—is a habit worth building from the very first session.

What is uv and Why Use It?

When you install Python, you get the language itself but not the scientific libraries listed above. Those must be installed separately, and managing them can become complicated quickly: different projects may need different versions of the same library, and installing everything into a single global environment is a reliable way to create hard-to-diagnose conflicts.

`uv` is a modern Python package and project manager that solves this cleanly.¹ It handles three things for you automatically:

1. Installing and managing Python itself

¹ `uv` is developed by Astral and is written in Rust, which makes it significantly faster than older tools like `pip` and `conda`. Full documentation is available at <https://docs.astral.sh/uv/>.

2. Creating isolated *virtual environments* for each project
3. Installing and tracking the exact versions of every package your project depends on

The concept of a virtual environment deserves a moment of attention before we continue.

What is a Virtual Environment?

A virtual environment is an isolated copy of Python and its installed packages that belongs to one specific project. Think of it like a clean laboratory bench: everything on that bench is exactly what that experiment needs, and nothing from a different experiment can interfere with it.

When you work inside a virtual environment, installing or upgrading a package affects only that project—not anything else on your computer. This makes your work reproducible: a collaborator can recreate your exact environment from a single configuration file and run your code without modification.

Installing *uv*

Open a terminal.² Run the appropriate command for your operating system:

macOS and Linux:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows (PowerShell):

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

After installation, close and reopen your terminal, then verify that *uv* is available:

```
uv --version
```

You should see output like `uv 0.4.x`. If you see a command not found error, consult the installation guide at <https://docs.astral.sh/uv/getting-started/installation/>.

Creating Your First Project

uv organises work around *projects*. A project is a folder that contains your code alongside a configuration file (`pyproject.toml`) that records which packages your project needs and in which versions.

² **macOS:** the Terminal application is in Applications/Utilities.

Windows: use PowerShell (search the Start menu). Windows Subsystem for Linux (WSL2) is an excellent alternative that provides a full Linux environment inside Windows—see <https://learn.microsoft.com/en-us/windows/wsl/> for setup instructions.

Linux: any terminal emulator will work.

Navigate to wherever you store your work and create a new project:

```
cd ~/Documents          # or wherever you prefer to
                        keep projects
uv init saxs-tutorial
cd saxs-tutorial
```

uv will create the following structure:

```
saxs-tutorial/
  pyproject.toml      # project configuration and
                      dependencies
  README.md           # project description (edit
                      this freely)
  hello.py            # a minimal starter script
```

What is pyproject.toml?

pyproject.toml is the central configuration file for your project. It records the project name, the Python version required, and the list of packages your project depends on. You will rarely need to edit it by hand—uv keeps it up to date automatically as you add packages. It is the file that allows anyone else (or a future you) to recreate your environment exactly.

Installing Scientific Packages

Install the packages we will need throughout this guide:

```
uv add numpy scipy matplotlib jupyterlab
```

uv creates a virtual environment in a hidden .venv folder inside your project, installs the requested packages into it, and updates pyproject.toml automatically. The first time you run this it may take a minute or two to download everything.

Verify the installation by asking uv to run a quick Python check:

```
uv run python -c "import numpy, scipy, matplotlib;
print('All packages OK!')"
```

If you see All packages OK, your environment is ready.

Try It Yourself

Open pyproject.toml in VS Code (we will install VS Code in Chapter I) and find the section that lists your installed packages. What version of NumPy was installed? How would you

find out what the latest available version is?

Version Control with Git and GitHub

Why Scientists Should Use Version Control

Imagine writing a simulation, obtaining results you are happy with, then making changes to improve the code—only to discover that the new version no longer reproduces the original results, and you cannot remember exactly what you changed. This is an extremely common and genuinely frustrating experience.

Version control is the solution. Git is a tool that records every change you make to your project files, along with a short description of what changed and why. At any point you can look back through the complete history of your work, compare any two versions side by side, or restore an earlier state exactly.

GitHub is a website that stores a copy of your Git history in the cloud, making it accessible from any computer and easy to share with collaborators or a supervisor.³

³ GitHub (<https://github.com>) is the most widely used hosting service in the scientific community, but alternatives exist—GitLab (<https://gitlab.com>) and Bitbucket (<https://bitbucket.org>) offer similar functionality.

The Core Git Workflow

The everyday Git workflow involves three steps that you will repeat throughout this guide and beyond:

1. **Stage** the changes you want to save: `git add`
2. **Commit** with a short message describing what changed:
`git commit`
3. **Push** your commit to GitHub: `git push`

Think of a commit as a labelled snapshot of your project at a specific moment. A good commit message is short but informative. `add sphere form factor function` is far more useful than `update or changes` when you are looking back through history six months later.

Installing Git

macOS: Git is often pre-installed. Check by running `git -version` in a terminal. If it is absent, install the Xcode Command Line Tools:

```
xcode-select --install
```

Alternatively, download Git directly from <https://git-scm.com/downloads>.

Windows: Download the installer from <https://git-scm.com/downloads/win> and accept the default options. Git Bash, which is included with this installer, provides a convenient terminal for the commands in this guide if you are not using WSL2.

Linux:

```
# Debian or Ubuntu
sudo apt install git

# Fedora
sudo dnf install git
```

Confirm the installation:

```
git --version
```

Configuring Git

Before using Git for the first time, tell it your name and email address. These are attached to every commit and will be visible in your project history:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Use the same email address you will register with GitHub.

Creating a GitHub Account

Go to <https://github.com> and create a free account if you do not already have one. Choose a professional username—it will be visible to collaborators and can appear on a CV or portfolio.

Setting Up SSH Authentication

GitHub needs a way to verify that it is really you when you push code to your account. The recommended method is SSH key authen-

tication, which is more secure than a password and, once configured, requires no further action from you.

Step 1: Generate an SSH key pair.

```
ssh-keygen -t ed25519 -C "your.email@example.com"
```

When prompted for a file location, press Enter to accept the default (`~/.ssh/id_ed25519`). When prompted for a passphrase, you may press Enter twice to leave it empty, or set a passphrase for additional security.⁴

⁴ On macOS, a passphrase can be stored in the system Keychain so you are not prompted repeatedly.

Public Keys and Private Keys

SSH authentication works by generating two mathematically linked files: a *private key* stored only on your computer (never shared) and a *public key* shared with GitHub. When you connect to GitHub, your computer proves it holds the private key without ever revealing it—similar to a lock-and-key system where the lock is public but the key is yours alone. The private key is stored at `~/.ssh/id_ed25519`. **Never share or upload your private key file under any circumstances.**

Step 2: Copy your public key.

```
# macOS
cat ~/.ssh/id_ed25519.pub | pbcopy

# Linux (copy the printed output manually)
cat ~/.ssh/id_ed25519.pub

# Windows PowerShell
Get-Content ~/.ssh/id_ed25519.pub | Set-Clipboard
```

Step 3: Add the public key to GitHub.

1. Log in to <https://github.com>
2. Click your profile picture (top right) and choose **Settings**
3. In the left sidebar, click **SSH and GPG keys**
4. Click **New SSH key**
5. Give it a descriptive title (e.g. *MacBook 2025*) and paste your public key into the Key field
6. Click **Add SSH key**

Step 4: Test the connection.

```
ssh -T git@github.com
```

A successful connection produces a message like:

Hi username! You've successfully authenticated, but GitHub does not provide shell access.

Full documentation for this process is available at <https://docs.github.com/en/authentication/connecting-to-github-with-ssh>.

Creating Your Repository and Connecting It Locally

Step 1: Create a new repository on GitHub.

1. Click the + icon (top right) and choose **New repository**
2. Name it `saxs-tutorial`
3. Add a short description if you like
4. Choose **Public** or **Private** according to your preference
5. Check **Add a README file**
6. Click **Create repository**

Step 2: Link your local project folder to GitHub.

Inside your `saxs-tutorial` folder, run:⁵

```
git init
git remote add origin git@github.com:yourusername/
saxs-tutorial.git
git branch -M main
```

⁵ Replace `yourusername` with your actual GitHub username.

You are now ready to push code from your computer to GitHub.

VS Code and Jupyter Notebooks

Installing VS Code

Visual Studio Code (VS Code) is a free, cross-platform code editor developed by Microsoft. It has first-class support for Python and Jupyter notebooks and is widely used in both academic research and industry. Download it from <https://code.visualstudio.com> and follow the installer for your operating system.

Installing Extensions

VS Code's capabilities are extended through *extensions*. Click the Extensions icon in the left sidebar (or press Ctrl+Shift+X on Windows/Linux, Cmd+Shift+X on macOS) and install:

- **Python** (by Microsoft) — language support, linting, and environment management
- **Jupyter** (by Microsoft) — run Jupyter notebooks directly inside VS Code

After installing the Python extension, open the Command Palette (Ctrl+Shift+P / Cmd+Shift+P) and search for Python: Select Interpreter. Choose the interpreter inside your `.venv` folder—it will appear as something like `.venv (saxs-tutorial)`. This tells VS Code to use the packages you installed with `uv`.

Opening Your Project

Go to **File** → **Open Folder** and select your `saxs-tutorial` folder. The file explorer on the left shows the project contents.

Anatomy of a Jupyter Notebook

A Jupyter notebook is a document made up of *cells*, of which there are two types you will use regularly:

- **Markdown cells** — formatted text, equations written in $\text{\LaTeX}$ notation, and narrative explanation
- **Code cells** — Python code that can be run interactively, one cell at a time, with output appearing directly below

This combination makes notebooks particularly well-suited to scientific work: you can describe what you are about to compute, run the code, and have the result—including figures—appear in the same document. A well-written notebook is a complete record of a calculation that a collaborator (or your future self) can follow and reproduce.

Hello, Scientific Python

Create a new file called `chapter_01_hello.ipynb`. In the VS Code file explorer, click the new-file icon, type the filename with the `.ipynb` extension, and VS Code will open it automatically as a notebook.

Click **+ Markdown** to add your first markdown cell and type:

```
# Chapter 1: Hello, Scientific Python

This notebook confirms that the environment is
working correctly.
```

Add a code cell and type the traditional first program:

```
1 print("Hello, world!")
```

Press Shift+Enter to run the cell. The text `Hello, world!` should appear immediately below it.

Now test the scientific packages by adding a second code cell:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 200 evenly spaced values from 0 to 2*pi
5 x = np.linspace(0, 2 * np.pi, 200)
6 y = np.sin(x)
7
8 fig, ax = plt.subplots()
9 ax.plot(x, y, color="steelblue", label=r"$\sin(x)$")
10 ax.set_xlabel(r"$x$ (radians)")
11 ax.set_ylabel(r"$\sin(x)$")
12 ax.legend(loc="upper right")
13 plt.show()
```

Run this cell. A sine wave should appear inline below the cell. If it does, your environment is fully working and you are ready to move on.

Inline Figures in VS Code

When running a plain `.py` script, matplotlib typically opens a figure in a separate window. Inside a Jupyter notebook in VS Code, figures appear inline, directly below the code cell that produced them. This is one of the reasons notebooks are convenient for exploratory scientific work—the code, the narrative, and the results all live together in one place.

Your First Commit

You have set up your environment, installed your packages, and written your first notebook. Before moving to the science, take a few minutes to save this work permanently using Git. This habit—committing at every meaningful checkpoint—is one of the most valuable practices you can build as a computational scientist.

Staging, Committing, and Pushing

Open a terminal inside VS Code by pressing the backtick key while holding Ctrl (or going to **Terminal** → **New Terminal**). You should be in your `saxs-tutorial` directory automatically.

Step 1: Check what Git can see.

```
git status
```

Git will list any new or modified files that have not yet been committed. Your notebook and `pyproject.toml` should appear here.

Step 2: Stage the files you want to include.

```
git add chapter_01_hello.ipynb pyproject.toml README
.md
```

Step 3: Commit with a clear message.

```
git commit -m "add hello world notebook and project
configuration"
```

Step 4: Push to GitHub.

```
git push -u origin main
```

The `-u origin main` flags only need to be included on the first push. After that, `git push` alone is sufficient.

Navigate to your repository on GitHub and refresh the page—your files should now be visible there.

Ignoring Files That Should Not Be Tracked

Some files in your project should never be committed to Git. The `.venv` folder, for instance, can be several hundred megabytes and can always be recreated from `pyproject.toml` using `uv sync`. Compiled Python files (`.pyc`) are also unnecessary to track.

Create a file called `.gitignore` in your project root:

```
.venv/
__pycache__/*
*.pyc
.DS_Store      # macOS folder metadata
```

Add this file to Git and commit it:

```
git add .gitignore
git commit -m "add gitignore for Python project"
git push
```

GitHub's Python Template

When creating a new repository on GitHub, there is an option to add a `.gitignore` automatically using a language-specific template. Choosing the **Python** template provides a comprehensive list of files to ignore and is a convenient starting point for new projects.

A Note on Commit Frequency

There is no single correct answer to how often you should commit. A useful rule of thumb: commit whenever you reach a state you would be unhappy to lose. In practice this might mean committing after writing a new function, after a plot looks the way you want it, or at the end of each working session.

Smaller, more frequent commits with descriptive messages are almost always preferable to large, infrequent commits labelled `stuff` or `final`.

Try It Yourself

1. Add a new markdown cell to your notebook that describes in your own words what the sine wave plot is showing.
2. Run the notebook to confirm everything still works.
3. Stage and commit the updated notebook with an appropriate message.

4. Push the commit to GitHub and confirm it appears on the website.

Remote Computing via SSH

This appendix is provided for reference and can safely be skipped for now. You will not need remote computing until later in the tutorial series. Return here when your supervisor or instructor indicates that it is time.

What is a Remote Computer and Why Use One?

Some calculations in later chapters will be computationally expensive. Rather than waiting for your laptop to finish a long job, you can connect to a more powerful remote machine—such as a departmental server or a high-performance computing cluster—and run the code there.

The standard tool for this is **SSH** (Secure Shell), which lets you open a terminal session on a remote machine over a network connection.

Connecting to a Remote Machine

```
ssh username@hostname.university.edu
```

Replace `username` with your account name on the remote system and `hostname.university.edu` with the address your institution provides.

If you have already set up an SSH key pair (Chapter I), you can copy your public key to the remote machine to avoid entering a password each time:

```
ssh-copy-id username@hostname.university.edu
```

Transferring Files

To copy a file *to* the remote machine:

```
scp localfile.ipynb username@hostname.university.edu  
:~/destination/
```

To copy a file *from* the remote machine to your local computer:

```
scp username@hostname.university.edu:~/remotefile.
ipynb ./
```

For whole folders, add the `-r` (recursive) flag:

```
scp -r saxs-tutorial/ username@hostname.university.
edu:~/
```

Running Jupyter Notebooks on a Remote Machine

You can run a Jupyter server on the remote machine and view it in your local browser by forwarding the network port over SSH.

Step 1: Connect with port forwarding enabled:

```
ssh -L 8888:localhost:8888 username@hostname.
university.edu
```

Step 2: On the remote machine, start JupyterLab:

```
uv run jupyter lab --no-browser --port=8888
```

Step 3: Copy the URL that JupyterLab prints (it will look like `http://localhost:8888/lab?token=...`) and paste it into your local browser.

You now have a full JupyterLab interface running on the remote machine, displayed in your local browser.

Keeping Sessions Alive with `tmux`

If your SSH connection drops while a calculation is running, that calculation will be killed. `tmux` is a terminal multiplexer that keeps your session alive on the remote machine even if the connection is interrupted.

```
# Start a new named session
tmux new -s tutorial

# Detach from the session (it keeps running in the
  background)
# Press Ctrl+B, then D

# Reattach after reconnecting via SSH
tmux attach -t tutorial
```

A concise `tmux` reference is available at <https://tmuxcheatsheet.com>.

Exercise Solutions

*Solutions will be added here progressively as the tutorial guide grows.
Attempt each exercise on your own before consulting this appendix.*

Chapter 1

Exercise 1.1

Open `pyproject.toml` in VS Code. The `[project]` section contains a `dependencies` key that lists each installed package with its version constraint. To see what the latest available version of NumPy is, run:

```
uv add numpy          # uv will report if a newer
                        version is available
pip index versions numpy # lists all available
                        versions
```

The `uv` documentation at <https://docs.astral.sh/uv/> describes how to upgrade packages to newer versions.

Chapter 4

Exercise 4.1

After adding your markdown cell and running the notebook, the staging and commit sequence is:

```
git add chapter_01_hello.ipynb
git commit -m "add description of sine wave plot to
               notebook"
git push
```

The commit message should be specific to what changed. If you also modified other files, include them in the `git add` command before committing.